



National Autonomous University of Mexico

Computer Engineering

Compilers

LEXER REPORT

STUDENT:
321117524
321147561
321652485
321030122
321310275

Group:
5
Semester:
2027-I

México City, México. October 2026



Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Motivation	3
1.3	Objectives	3
2	Theoretical Framework	4
2.1	Lexical Analyzer	4
2.2	Token, Lexeme, and Pattern	4
2.3	Regular Expressions	5
2.4	Finite Automata	5
2.5	Longest Match and Rule Priority	5
2.6	Elements That Do Not Generate Tokens	6
2.7	Lexical Errors	6
2.8	The re2c Generator	6
2.9	Context-Free Grammars	6
2.10	Left Factoring	7
3	Lexical Analyzer Implementation	7
3.1	Design methodology and conventions	7
3.1.1	Overview of the development	8
3.2	Finite automata design	8
3.2.1	Keywords	9
3.2.2	Identifiers	9
3.2.3	Operators	10
3.2.4	Punctuation	10
3.2.5	Numeric constants	10
3.2.6	String literals	11
3.2.7	Whitespace	11
3.2.8	Identifier starting with a digit	11
3.2.9	Identifier starting with a symbol	11
3.2.10	Symbol inside an identifier	11
3.2.11	Symbol at the end of an identifier	12
3.3	Regular expressions	12
3.4	Rule priority and conflict resolution	13
3.5	Lexer implementation	14
3.5.1	Architecture and modules	14
3.5.2	Token model	15
3.5.3	Scanner function	16
3.5.4	Main program and output format	16
3.5.5	Token counting	18
3.5.6	Build and execution	19
3.6	Context-Free Grammar proposal	19



4	Results	20
4.1	Test environment	20
4.2	Test 1: Reading examples	21
4.3	Test 2: Whitespace handling	21
4.4	Test 3: Keywords versus identifiers	22
4.5	Test 4: Constants	23
4.6	Test 5: Operators	24
4.7	Test 6: Lexical errors	25
4.8	Test 7: Project test file	26
5	Conclusions	28



1 Introduction

1.1 Problem Statement

Lexical analysis is the first phase of the compilation process. At this stage, the source program reaches the compiler as an undifferentiated sequence of characters, and it must be divided into the smallest meaningful units of the language, known as tokens. These units include keywords, identifiers, constants, literals, operators, and punctuation symbols. Every later phase of the compiler works on this sequence of tokens instead of on raw characters.

The problem addressed in this project is the construction of a lexical analyzer for a subset of the C language. The analyzer receives a text string or the contents of a file, classifies each element of the input into its corresponding token category, and reports the resulting sequence of tokens along with their total count. It must also detect lexical errors, such as malformed identifiers and characters that do not belong to the language, without interrupting the analysis. As a reference, the analyzer is tested with simple C statements such as:

```
printf("This is an example");  
int a = 10;
```

1.2 Motivation

The lexical analyzer is the bridge between the text written by a programmer and the structures that a compiler can process. A correct lexical analysis simplifies the rest of the compiler: the syntactic analyzer does not need to deal with blanks, line breaks, or the spelling of each word, because it receives a clean stream of classified symbols.

Building a lexical analyzer also provides a concrete way to apply the theory of regular languages. Regular expressions and finite automata stop being abstract notations and become tools that decide, for every character of the input, which token is being formed. Furthermore, it makes clear why rules such as the longest match and the order of the patterns are necessary to classify the input without ambiguity.

1.3 Objectives

The main objective is to develop a lexical analyzer capable of recognizing the tokens of a subset of the C language, classifying them, and counting them, using regular expressions and finite automata as its theoretical foundation.

The specific objectives are:

- Design a finite automaton and a regular expression for each token category: keywords, identifiers, constants, literals, operators, and punctuation.



- Implement the analyzer in separate modules, so that the lexical rules remain independent from the handling of input and output.
- Accept the source code from a file or from a text string.
- Display the sequence of classified tokens and the total number of tokens found.
- Detect and report lexical errors, specifically invalid identifiers and invalid characters.
- Propose a basic Context-Free Grammar for one of the analyzed statements and improve it through left factoring, in order to show how the tokens produced by the lexer are used by the following phase.

2 Theoretical Framework

2.1 Lexical Analyzer

A lexical analyzer, also called a lexer or scanner, is the first phase of a compiler. It reads the source program character by character, groups the characters into meaningful sequences called lexemes, and produces a token for each one of them. Besides this main task, the lexer removes comments and whitespace that are irrelevant for the following phases. The syntactic analyzer then requests tokens from the lexer one at a time, as it needs them [1].

2.2 Token, Lexeme, and Pattern

A token is an abstract symbol that represents a category, such as identifier or operator. A lexeme is the concrete sequence of characters matched in the source code, for example `counter`. A pattern is the rule, usually a regular expression, that describes all the lexemes that belong to a token.

The categories used in this project are the following:

- **Keywords:** reserved words of the language.
- **Identifiers:** names given to variables and functions.
- **Constants:** numeric values, either integers or numbers with a decimal part and an exponent.
- **Literals:** sequences of characters enclosed in double quotation marks.
- **Operators:** arithmetic, relational, and assignment operations.
- **Punctuation:** symbols that structure the program, such as parentheses, braces, commas, and semicolons.



- **Errors:** invalid identifiers and invalid characters.

In this project, the attribute is the specific subtype of the token (for example, which operator or which keyword was found), and it takes the value -1 for identifiers, constants, and literals.

2.3 Regular Expressions

A regular expression is a formal notation that describes a set of strings by means of three operations: concatenation, alternation, and repetition. The lexical structure of programming languages can be described with regular languages, so each token category can be defined by a regular expression.

For example, an identifier that starts with a letter and continues with letters or digits is written as:

$$\text{letter}(\text{letter} \mid \text{digit})^*$$

2.4 Finite Automata

A finite automaton is an abstract machine that reads a string one symbol at a time and moves between a finite number of states. The string is accepted if the machine finishes in an accepting state.

Any regular expression can be converted into a non-deterministic finite automaton (NFA), and this can be transformed into a deterministic finite automaton (DFA), which has exactly one transition for each state and symbol and can therefore be executed efficiently. The transformation from an NFA to a DFA is performed using the subset construction, and the resulting DFA can be minimized [3].

In essence, a lexer is a DFA that recognizes the union of the patterns of all the token categories.

2.5 Longest Match and Rule Priority

When several patterns can match the input at the same position, the lexer applies two rules. First, the pattern that matches the longest prefix is selected (maximal munch), so that `==` is read as a single operator and not as two assignment operators.

Second, if two patterns match the same length, the one defined first prevails. This is why `int` is classified as a keyword and not as an identifier, while a word such as `integer` is an identifier, since it matches the identifier pattern with a longer length.



2.6 Elements That Do Not Generate Tokens

Blanks, tabulations, and line breaks only separate lexemes. The lexer consumes them but does not pass them to the following phase.

2.7 Lexical Errors

A lexical error occurs when a sequence of characters does not match any valid pattern, for instance an illegal character or a malformed identifier. Because the lexer only sees a small, local part of the source, few errors can be detected at this stage. For example, a misspelled keyword is still a valid identifier, so it cannot be recognized as an error by the lexer. A common recovery strategy is to discard characters until a well-formed token is found. In this project, the lexer instead reports the offending sequence as an error token, with a specific error type, and continues the analysis, so that all the errors of the input can be found in a single execution.

2.8 The re2c Generator

re2c is a lexer generator that takes a set of regular expressions, each one associated with an action, and produces C or C++ code that implements the equivalent deterministic automaton [2, 4]

The generated scanner applies the longest match rule and gives priority to the first rule in case of a tie. In this project, the automata and regular expressions are designed first, and re2c is used to translate that design into executable code.

2.9 Context-Free Grammars

A Context-Free Grammar (CFG) is defined by a quadruple

$$G = (V, \Sigma, S, P)$$

where V is the set of non-terminal symbols, Σ is the set of terminal symbols, $S \in V$ is the start symbol, and P is the set of production rules.

The tokens generated by the lexer are the terminal symbols of the grammar used by the syntactic analyzer, which makes the CFG the natural continuation of the lexical analysis.



2.10 Left Factoring

When two or more productions of the same non-terminal begin with the same prefix, a top-down parser cannot decide which one to apply by looking only at the next token. Left factoring solves this problem by grouping the common prefix in a single production and creating a new non-terminal for the different endings.

For example, the rules

$$A \rightarrow \alpha\beta \mid \alpha\gamma$$

can be transformed into:

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta \mid \gamma \end{aligned}$$

This transformation is different from the elimination of left recursion, which deals with productions of the form:

$$A \rightarrow A\alpha$$

3 Lexical Analyzer Implementation

This section describes how the lexical analyzer was built. It covers the design methodology, the finite automata and regular expressions, the implementation details, and the context-free grammar proposal.

3.1 Design methodology and conventions

The development followed two stages. First, the theoretical design was carried out: a finite automaton was defined for each token category and a regular expression was derived from each automaton. Second, the implementation was done in C++. The analyzer recognizes tokens of a subset of the C language.

The automata were drawn using a finite automata drawing tool. In the figures, the initial state is marked with an incoming arrow, and accepting states are drawn with a double circle. The transition labels use the following symbol classes:

- `letter` = A-Z and a-z
- `digit` = 0-9
- `symbol` = " , ' , @ , # , \$, % , ? , and the dot
- `whitespace` = space, tab, carriage return, and line feed

3.1.1 Overview of the development

The lexical analyzer is implemented in C++ and recognizes the following token categories:

- Keywords
- Identifiers
- Constants
- Literals
- Operators
- Punctuation
- Errors

Table 1 summarizes these categories.

Category	Description
Keywords	Reserved words: <code>if</code> , <code>else</code> , <code>char</code> , <code>int</code> , <code>float</code> , <code>return</code> .
Identifiers	Names for variables and functions. Start with a letter, continue with letters or digits.
Constants	Numeric values: integers, decimals, and numbers with an exponent.
Literals	Strings enclosed in double quotation marks.
Operators	Arithmetic, relational, and assignment operators.
Punctuation	Symbols that structure the program: <code>() { } , ;</code> .
Errors	Invalid identifiers and invalid characters.

Table 1: Token categories recognized by the lexical analyzer.

The rest of this section is organized as follows. Section 3.2 presents the finite automata for each category. Section 3.3 lists the regular expressions. Section 3.4 explains the rule priority and how conflicts are resolved. Section 3.5 describes the implementation, including the architecture, the token model, the scanner function, the main program, and the token counting. Finally, Section 3.6 proposes a context-free grammar for one of the analyzed statements.

3.2 Finite automata design

Notation. letter = [A-Za-z], digit = [0-9], symb = {", ' , @, #, \$, %, ?, .}.

3.2.1 Keywords

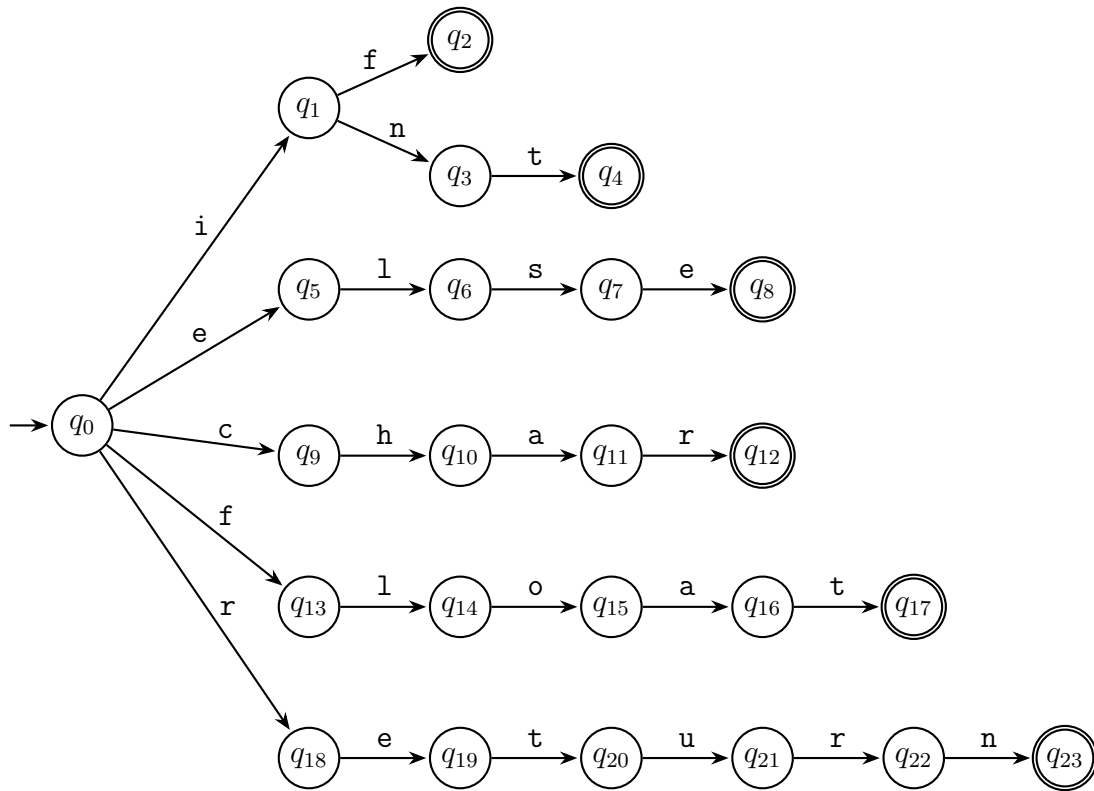


Figure 1: Keywords automaton: if, int, else, char, float, return.

3.2.2 Identifiers

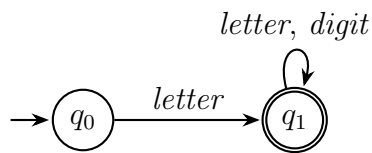


Figure 2: Identifiers automaton: $letter (letter \mid digit)^*$.

3.2.3 Operators

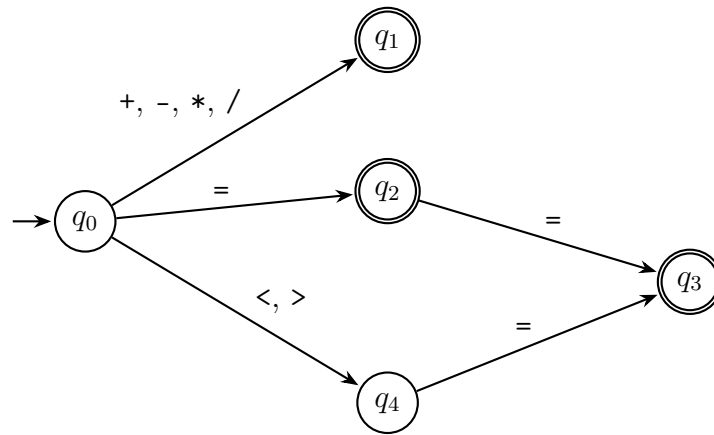


Figure 3: Operators automaton. Accepts $(+ \ - \ * \ /)$, $=$, $==$, $>=$ and $<=$.

3.2.4 Punctuation

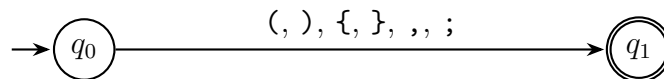


Figure 4: Punctuation automaton: $(\) \{ \} \ , \ ;$.

3.2.5 Numeric constants

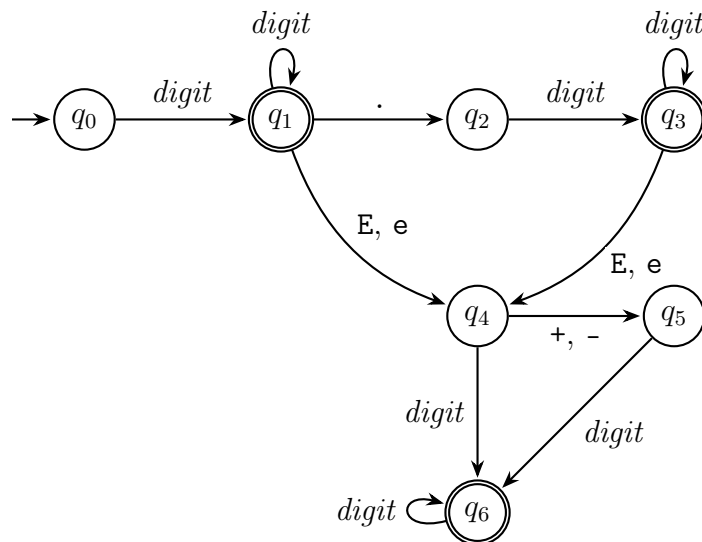


Figure 5: Numeric constants automaton: $digit^+ (. digit^+)? ([Ee][+-]? digit^+)?$.

3.2.6 String literals

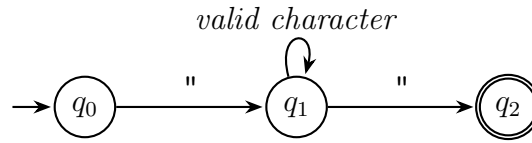


Figure 6: String literals automaton: " (*valid character*)^{*} ". A valid character is any *letter*, *digit*, operator, whitespace, punctuation mark or one of ' @ # \$ % ? .

3.2.7 Whitespace

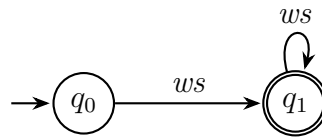


Figure 7: Whitespace automaton: *ws* is a space, tab, \r or \n.

3.2.8 Identifier starting with a digit

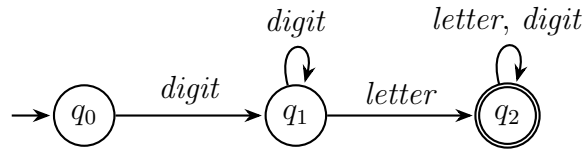


Figure 8: Error automaton ERROR_ID_START_DIGIT: *digit*⁺ *id*.

3.2.9 Identifier starting with a symbol

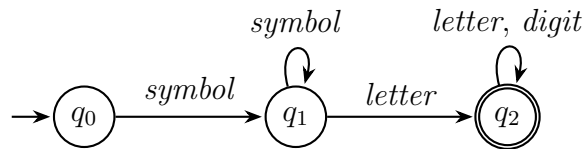


Figure 9: Error automaton ERROR_ID_START_SYMBOL: *symbol*⁺ *id*.

3.2.10 Symbol inside an identifier

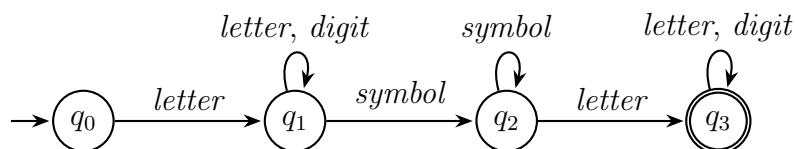


Figure 10: Error automaton ERROR_ID_MIDDLE_SYMBOL: *id symbol*⁺ *id*.

3.2.11 Symbol at the end of an identifier

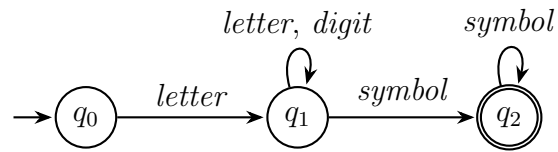


Figure 11: Error automaton `ERROR_ID_LAST_SYMBOL`: $id\ symbol^+$.

3.3 Regular expressions

Name	Definition
letter	[A-Za-z]
digit	[0-9]
id	letter (letter digit)*
operator	+ - * / == >= <= =
symbol	" ' @ # \$ % ? .
valid_char	letter digit operator punctuation ws ' @ # \$ % ? .

Table 2: Auxiliary definitions used by the regular expressions.



Category	Regular expression (standard notation)
Keyword	<code>if else char int float return</code>
Identifier	<code>letter (letter digit)*</code>
Constant	<code>digit+ ("." digit+)? ([Ee] [+ -]? digit+)?</code>
Literal	<code>" valid_char* "</code>
Arithmetic operator	<code>+ - * /</code>
Relational operator	<code>== >= <=</code>
Assignment operator	<code>=</code>
Punctuation	<code>() { } , ;</code>
Whitespace (ignored)	<code>[\t\r\n]+</code>
Invalid id: start digit	<code>digit+ id</code>
Invalid id: start symbol	<code>symbol+ id</code>
Invalid id: middle symbol	<code>id symbol+ id</code>
Invalid id: last symbol	<code>id symbol+</code>
Invalid character	<code>any other character</code>
End of input	<code>\0</code>

Table 3: Regular expressions for each token category of the lexer.

3.4 Rule priority and conflict resolution

The lexer uses two rules when the lexeme has aliasing, the first is the longest prefix, and in case of a tie it uses the first declared.



Priority	Rule
1	White spaces
2	Literals
3	Keywords
4	Operators
5	Constants
6	Punctuation
7-10	Invalid identifier types
11	Identifiers
12	Null character
13	Non recognized characters

Table 4: Priority of the rules

For example, if the lexer recognizes the lexeme `int` it could be tokenized as a keyword or identifier, but the keyword rule was declared before the identifier rule, so it assigned as keyword.

3.5 Lexer implementation

3.5.1 Architecture and modules

The lexer works with five archives, each one of them has a different responsibility, making a modularized program with the next classification:

Archive	Description
<code>lexer.re</code>	This is the lexer itself, contains the grammar and all the rules to scan and classify the lexemes into tokens.
<code>token.h</code>	Defines the token's and attributes for them if it has
<code>token.cpp</code>	Implements the functions that turn lexemes into values and tokens into strings
<code>main.cpp</code>	Reads the input from the <code>cadena_de_prueba.txt</code> file and uses the scanner for tokenization, then prints the output
<code>cadena_de_prueba.txt</code>	It contains the input for the lexer.

Table 5: Description of the project archives



3.5.2 Token model

The token model is defined in `token.h` and `token.cpp`. It contains the enumerations and functions that represent and classify every token.

The main enumeration is `TokenType`, which lists the general categories:

- `TOKEN_KEYWORD`
- `TOKEN_IDENTIFIER`
- `TOKEN_PUNCTUATION`
- `TOKEN_OPERATOR`
- `TOKEN_CONSTANT`
- `TOKEN_LITERAL`
- `TOKEN_ERROR`

For tokens that have a subtype, additional enumerations are defined. Keywords use `TokenKeyword` (for example `KEYWORD_IF`, `KEYWORD_INT`). Punctuation uses `TokenPunctuation` (for example `PUNCTUATION_LPAREN`, `PUNCTUATION_SEMICOLON`). Operators use `TokenOperator` (for example `OPERATOR_PLUS`, `OPERATOR_ASSIGN`). Errors use `TokenError` (for example `ERROR_ID_START_DIGIT`, `ERROR_INVALID_CHAR`).

To get the subtype from a lexeme, some helper functions are implemented:

- `getKeywordValue` returns the `TokenKeyword` for a keyword.
- `getPunctuationValue` returns the `TokenPunctuation`.
- `getOperatorValue` returns the `TokenOperator`.
- `errorToString` returns a string for an error value.

Also, functions like `tokenToString`, `keywordToString`, `punctuationToString` and `operatorToString` convert the enumeration values to strings. This is used when printing the tokens, so the output shows names like `TOKEN_KEYWORD` or `KEYWORD_INT` instead of numbers.

The scanner only sets the `TokenType`, an optional subtype, and the lexeme. The main program then uses these helper functions to print the tokens.



3.5.3 Scanner function

The scanner function is declared inside the `lexer.re` archive, it works with `re2c`. Its signature is:

```
bool yylex(const char *&YYCURSOR, TokenType &token, TokenError
           &errorValue, string &lexeme)
```

- `*YYCURSOR` is a pointer to a char, it iterates the string and consume the characters and works as a reference to return the content of the token.
- `token` is the type of the token, it is also a reference value for the main function, making it possible to classify the token.
- `errorValue` is the type of error in case it is recognized
- `lexeme` is the value of the token, the string that is tokenized.

This is where the previous regular expressions are declared, such as:

```
letter = [A-Za-z];
digit = [0-9];

keyword = "if"|"else"|"char"|"int"|"float"|"return";
...
id = letter(letter | digit)*;
number = digit+("." digit+)?([Ee][+-]? digit+)?;
...
```

For each pattern it has a block that executes when it is recognized, for example the block for keywords is:

```
keyword {
    lexeme.assign(start, YYCURSOR - start);
    token = TOKEN_KEYWORD;
    return true;
}
```

The cursor moves following the next rules:

1. All white spaces are ignored.
2. Searches for the largest lexeme, when it is recognized, the pointer goes forward to the end of the lexeme and return the token.
3. If the pointer reaches the null character returns null and finish the scan.

3.5.4 Main program and output format

The main program is implemented in `main.cpp`. Its purpose is to read the input, call the scanner repeatedly, complete the attribute of each token, and print the results in a readable format.



Input handling. The input is read from a plain text file named `cadena_de_prueba.txt`. The function `openFile` opens the file and loads its entire content into a `string`. If the file cannot be opened, it prints an error message and returns an empty string. Once the content is loaded, a pointer to the first character is obtained with `input.c_str()`. This pointer, named `YYCURSOR`, is the cursor that the scanner advances as it consumes characters.

Main loop. Before the loop, `printTokenTableHeader` prints the header of the token table. This header has three columns: `TOKEN`, `TYPE`, `SUBTYPE`, and `LEXEME`. The columns are aligned with `setw` so that each row is easy to read. The scanner function `yylex` is called inside a `while` loop. Its signature is:

```
bool yylex(const char *&YYCURSOR, TokenType &token, TokenError
           &errorValue, string &lexeme)
```

The function returns `true` when a token is recognized and `false` when the end of the input is reached. On each call, it updates `YYCURSOR`, sets the token category in `token`, sets `errorValue` if the token is an error, and stores the matched text in `lexeme`.

For every token returned, the program builds a structure of type `pair<int, pair<int, string>>`. The first integer is the token category (`TokenType`). The second integer is the subtype value, or `-1` if the category has no subtype. The string is the lexeme.

The subtype is obtained through the helper functions of the token model. For example, when the token is a keyword, the program calls `getKeywordValue` to convert the lexeme into a `TokenKeyword` value. The same happens with punctuation and operators. For errors, the subtype is already provided by the scanner in `errorValue`.

Output format. Each token is printed by the function `printToken` as a row of the table. The row shows the category name, the subtype name, and the lexeme. The category name is obtained with `tokenToString`. The subtype name is obtained with the corresponding conversion function (`keywordToString`, `punctuationToString`, `operatorToString`, or `errorToString`). When the category has no subtype (identifiers, constants, and literals), the program prints `NO_SUBTYPE_VALUE` instead of a number.

Table 6 shows an example row for each category, using the same column layout that the program produces.



TOKEN TYPE	SUBTYPE	LEXEME
TOKEN_KEYWORD	KEYWORD_INT	int
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	a
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	10
TOKEN_LITERAL	NO_SUBTYPE_VALUE	"This is an example"
TOKEN_OPERATOR	OPERATOR_ASSIGN	=
TOKEN_PUNCTUATION	PUNCTUATION_SEMICOLON	;
TOKEN_ERROR	ERROR_ID_LAST_SYMBOL	efmemf.

Table 6: Example row for each token category, using the same columns produced by `printToken`.

In the real output, the three columns are left-aligned using `setw`: 25 characters for the category, 30 for the subtype, and 30 for the lexeme. A line of dashes separates the header from the rows.

After the loop finishes, the program calls `printTokenCounters`. This function prints a second table with the count of tokens per category. The details of that table are described in the Token counting section. The main program itself only passes the six counters that it increments inside the loop.

With this design, the program can analyze a file and present the results in two clear tables: one for the tokens and one for the counts. This makes it easy to check that the scanner is working correctly and to detect lexical errors in the input.

3.5.5 Token counting

When a token is recognized, the corresponding counter increases. For example, the counting process for keyword tokens is:

```
case TOKEN_KEYWORD:
    token.second = {
        getKeywordValue(lexeme),
        lexeme
    };

    keywordCount++;
    break;
```

A similar block is defined for each category. Also, the program has a function for printing the token breakdown and the total count of valid tokens.



3.5.6 Build and execution

The build of the lexer is simple. The first step is to be located in the directory of the project. After that, compile the `lexer.re` file with `re2c`, that generates the `lexer.cpp` file.

```
re2c lexer.re -o lexer.cpp
```

Once it has been generated, the next step is compile all the files to create the executable lexer, this is possible with the following command:

```
g++ lexer.cpp main.cpp token.cpp -o test
```

Once done, it is possible to execute the lexer with the command:

```
./test
```

3.6 Context-Free Grammar proposal

The regular expressions defined in the previous section specify how each token of the language is written. Building on them, the proposed context-free grammar takes those tokens (keywords, identifiers, constants, literals, operators and punctuation marks) as its terminal symbols and describes how they can be combined to form valid declarations, statements and expressions.

The grammar is $G = (N, T, P, S)$ with start symbol $S = \text{program}$.

Terminals (T): `if`, `else`, `int`, `float`, `char`, `return`, `+`, `-`, `*`, `/`, `==`, `>=`, `<=`, `=`, `(`, `)`, `{`, `}`, `,`, `;`, and the token classes `id` (identifier), `num` (constant) and `str` (literal).

Non-terminals (N): `program`, `decl_list`, `decl`, `type`, `var_decl`, `init_list`, `init`, `func_decl`, `params_opt`, `params`, `param`, `block`, `stmt_list`, `stmt`, `matched`, `unmatched`, `other_stmt`, `expr`, `relop`, `arith`, `term`, `factor`, `args_opt`, `args`.

Productions (P):



```

program → decl_list
decl_list → decl_list decl | decl
decl → var_decl | func_decl

type → int | float | char
var_decl → type init_list ;
init_list → init_list , init | init
init → id | id = expr

func_decl → type id ( params_opt ) block
params_opt → params | ε
params → params , param | param
param → type id

block → { stmt_list }
stmt_list → stmt_list stmt | ε
stmt → matched | unmatched
matched → if ( expr ) matched else matched
          | other_stmt
unmatched → if ( expr ) stmt
           | if ( expr ) matched else unmatched
other_stmt → var_decl | id = expr ;
           | return expr ; | return ; | block

expr → arith relop arith | arith
relop → == | >= | <=
arith → arith + term | arith - term | term
term → term * factor | term / factor | factor
factor → ( expr ) | - factor | id ( args_opt )
        | id | num | str
args_opt → args | ε
args → args , expr | expr

```

4 Results

4.1 Test environment

The lexical analyzer was developed using C++23 and re2c version 4.6. re2c was used to generate the source code corresponding to the lexical analysis process.

For each test, the `lexer.re` file was first processed with re2c to generate the `lexer.cpp` file. Then, the generated file was compiled together with the remaining source files



using C++23.

4.2 Test 1: Reading examples

This test verifies the general behavior of the lexical analyzer using two expressions that contain different types of tokens. The purpose is to verify that every lexeme is correctly recognized and classified according to the rules defined in the lexer.

Together, both expressions produce a total of 10 valid tokens: 2 identifiers, 4 punctuation symbols, 1 literal, 1 keyword, 1 operator, and 1 constant.

Additionally, the expression `int a = 10;` produces the five expected tokens: a keyword, an identifier, an operator, a constant, and a punctuation symbol.

Input

```
printf("This is an example");
int a = 10;
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
TOKEN TYPE          SUBTYPE          LEXEME
-----
TOKEN_IDENTIFIER    NO_SUBTYPE_VALUE printf
TOKEN_PUNCTUATION   PUNCTUATION_LPAREN (
TOKEN_LITERAL       NO_SUBTYPE_VALUE  "This is an example"
TOKEN_PUNCTUATION   PUNCTUATION_RPAREN )
TOKEN_PUNCTUATION   PUNCTUATION_SEMICOLON ;
TOKEN_KEYWORD       KEYWORD_INT       int
TOKEN_IDENTIFIER    NO_SUBTYPE_VALUE  a
TOKEN_OPERATOR      OPERATOR_ASSIGN   =
TOKEN_CONSTANT      NO_SUBTYPE_VALUE  10
TOKEN_PUNCTUATION   PUNCTUATION_SEMICOLON ;

TOKEN TYPE          COUNT
-----
TOKEN_KEYWORD       1
TOKEN_IDENTIFIER    2
TOKEN_PUNCTUATION   4
TOKEN_OPERATOR      1
TOKEN_CONSTANT      1
TOKEN_LITERAL       1
-----
TOTAL VALID TOKENS  10
```

Figure 12: Output of Test 1.

4.3 Test 2: Whitespace handling

This test verifies that whitespace characters used to separate the elements of the input do not generate tokens. Different amounts of spaces, tabs, and line breaks are



included between the lexemes.

The lexical analyzer must ignore all these separation characters and produce only the tokens corresponding to the instruction `int a = 10;`. Therefore, the distribution of whitespace must not affect either the classification of the lexemes or the number of tokens obtained.

Input

```
int a =
    10 ;
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
```

TOKEN TYPE	SUBTYPE	LEXEME

TOKEN_KEYWORD	KEYWORD_INT	int
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	a
TOKEN_OPERATOR	OPERATOR_ASSIGN	=
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	10
TOKEN_PUNCTUATION	PUNCTUATION_SEMICOLON	;

TOKEN TYPE	COUNT

TOKEN_KEYWORD	1
TOKEN_IDENTIFIER	1
TOKEN_PUNCTUATION	1
TOKEN_OPERATOR	1
TOKEN_CONSTANT	1
TOKEN_LITERAL	0

TOTAL VALID TOKENS	5

Figure 13: Output of Test 2.

4.4 Test 3: Keywords versus identifiers

This test verifies the difference between keywords and identifiers. Strings that exactly match a reserved word of the language must be classified as `TOKEN_KEYWORD`.

On the other hand, when a keyword only appears as the prefix of a longer string, the complete string must be recognized as an identifier. Therefore, `int`, `if`, and `return` are recognized as keywords, while `integer`, `iff`, and `returns` are recognized as identifiers.

This test also verifies that the lexer recognizes the complete lexeme before assigning its corresponding token type.



Input

```
int integer if iff return returns
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
TOKEN TYPE          SUBTYPE          LEXEME
-----
TOKEN_KEYWORD       KEYWORD_INT      int
TOKEN_IDENTIFIER    NO_SUBTYPE_VALUE integer
TOKEN_KEYWORD       KEYWORD_IF       if
TOKEN_IDENTIFIER    NO_SUBTYPE_VALUE iff
TOKEN_KEYWORD       KEYWORD_RETURN   return
TOKEN_IDENTIFIER    NO_SUBTYPE_VALUE returns

TOKEN TYPE          COUNT
-----
TOKEN_KEYWORD       3
TOKEN_IDENTIFIER    3
TOKEN_PUNCTUATION   0
TOKEN_OPERATOR      0
TOKEN_CONSTANT      0
TOKEN_LITERAL       0
-----
TOTAL VALID TOKENS  6
```

Figure 14: Output of Test 3.

4.5 Test 4: Constants

This test verifies the recognition of the different numeric representations accepted by the lexical analyzer. Integers, decimal numbers, and numbers written using exponential notation are included.

The values 10, 92392.132, 3.14E-2, 5e3, and 1e5 must each be recognized as a single constant token.

The case 5. is also tested. According to the regular expression defined for numeric constants, at least one digit must appear after the decimal point. Therefore, 5 is recognized as a constant and the remaining period is reported as an invalid character.

Input

```
10 92392.132 3.14E-2 5e3 1e5 5.
```



Output

```
[javi@Javi-Arch Compilers]$ ./main
```

TOKEN TYPE	SUBTYPE	LEXEME
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	10
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	92392.132
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	3.14E-2
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	5e3
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	1e5
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	5
TOKEN_ERROR	ERROR_INVALID_CHAR	.

TOKEN TYPE	COUNT
TOKEN_KEYWORD	0
TOKEN_IDENTIFIER	0
TOKEN_PUNCTUATION	0
TOKEN_OPERATOR	0
TOKEN_CONSTANT	6
TOKEN_LITERAL	0
TOTAL VALID TOKENS	6

Figure 15: Output Test 4.

4.6 Test 5: Operators

This test verifies the recognition of the different operators defined by the language. It includes two-character relational operators, the assignment operator, and arithmetic operators.

The operators `==`, `>=`, and `<=` must each be recognized as a single token and associated with their corresponding subtype. Similarly, the operators `=`, `+`, `-`, `*`, and `/` must be correctly classified.

Finally, the symbol `<` is included by itself. Since this symbol is not currently defined as a valid operator on its own, it must be reported as an invalid character. The lexical analysis must then continue normally with the remaining lexemes.

Input

```
a == b >= c <= d = e + f - g * h / i
a < b
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
```

TOKEN TYPE	SUBTYPE	LEXEME
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	a
TOKEN_OPERATOR	OPERATOR_EQUAL	=
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	b
TOKEN_OPERATOR	OPERATOR_GREATER_EQUAL	≥
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	c
TOKEN_OPERATOR	OPERATOR_LESS_EQUAL	≤
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	d
TOKEN_OPERATOR	OPERATOR_ASSIGN	=
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	e
TOKEN_OPERATOR	OPERATOR_PLUS	+
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	f
TOKEN_OPERATOR	OPERATOR_MINUS	-
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	g
TOKEN_OPERATOR	OPERATOR_MULT	*
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	h
TOKEN_OPERATOR	OPERATOR_DIV	/
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	i
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	a
TOKEN_ERROR	ERROR_INVALID_CHAR	<
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	b

TOKEN TYPE	COUNT
TOKEN_KEYWORD	0
TOKEN_IDENTIFIER	11
TOKEN_PUNCTUATION	0
TOKEN_OPERATOR	8
TOKEN_CONSTANT	0
TOKEN_LITERAL	0

TOTAL VALID TOKENS	19
--------------------	----

Figure 16: Output of Test 5.

4.7 Test 6: Lexical errors

This test verifies the detection and classification of the different lexical errors considered by the analyzer. Each invalid lexeme must generate a `TOKEN_ERROR` together with its corresponding error subtype.

The test includes an identifier that starts with a digit, an identifier that starts with an invalid symbol, an identifier that contains an invalid symbol in the middle, and an identifier that ends with an invalid symbol. An invalid character is also included independently.

In addition, an unterminated literal is tested. Since the sequence does not contain a closing quotation mark, it does not satisfy the definition of a valid literal and must not be recognized as a single `TOKEN_LITERAL`. Instead, the invalid portion is reported according to the lexical rules and the analyzer continues processing the remaining input.



This test also verifies that a lexical error does not stop the analysis and that the lexer continues processing the following lexemes.

Input

```
1212msadas
@hola
ho@la
hola@
@
printf("Hola mundo);
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
TOKEN TYPE          SUBTYPE          LEXEME
-----
TOKEN_ERROR          ERROR_ID_START_DIGIT  1212msadas
TOKEN_ERROR          ERROR_ID_START_SYMBOL @hola
TOKEN_ERROR          ERROR_ID_MIDDLE_SYMBOL ho@la
TOKEN_ERROR          ERROR_ID_LAST_SYMBOL  hola@
TOKEN_ERROR          ERROR_INVALID_CHAR    @
TOKEN_IDENTIFIER     NO_SUBTYPE_VALUE     printf
TOKEN_PUNCTUATION    PUNCTUATION_LPAREN   (
TOKEN_ERROR          ERROR_ID_START_SYMBOL "Ho la
TOKEN_IDENTIFIER     NO_SUBTYPE_VALUE     mundo
TOKEN_PUNCTUATION    PUNCTUATION_RPAREN   )
TOKEN_PUNCTUATION    PUNCTUATION_SEMICOLON ;

TOKEN TYPE          COUNT
-----
TOKEN_KEYWORD        0
TOKEN_IDENTIFIER     2
TOKEN_PUNCTUATION    3
TOKEN_OPERATOR        0
TOKEN_CONSTANT       0
TOKEN_LITERAL        0
-----
TOTAL VALID TOKENS   5
```

Figure 17: Output of Test 6.

4.8 Test 7: Project test file

The final test uses the test file employed during the development of the project. Unlike the previous tests, this file combines literals, identifiers, constants, operators, and lexical errors in the same input.

The purpose of this test is to verify the behavior of the analyzer when valid and invalid lexemes are processed consecutively. According to the expected execution, the file produces a total of 15 tokens, of which 13 are valid tokens and 2 are lexical errors.



One interesting case is `efmemf.2121`. According to the rules defined for invalid identifiers and numeric constants, this input is divided into an error token followed by a constant.

The test also contains consecutive double quotation marks. In this case, the first quotation mark may close one literal while the following quotation mark starts another one, depending on the literal recognition rules defined in the lexical analyzer.

Finally, expressions containing the `+` operator verify that consecutive tokens can be correctly recognized even when no whitespace appears between them.

Input

```
"HOLA"  
como  
estas  
92392.132  
efmemf.2121  
1212msadas  
"h' dsds "" 'o"  
hola+hola  
hola+" 'sjas"
```

Output

```
[javi@Javi-Arch Compilers]$ ./main
```

TOKEN TYPE	SUBTYPE	LEXEME

TOKEN_LITERAL	NO_SUBTYPE_VALUE	"HOLA"
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	como
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	estas
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	92392.132
TOKEN_ERROR	ERROR_ID_LAST_SYMBOL	efmemf.
TOKEN_CONSTANT	NO_SUBTYPE_VALUE	2121
TOKEN_ERROR	ERROR_ID_START_DIGIT	1212msadas
TOKEN_LITERAL	NO_SUBTYPE_VALUE	"h' dsds "
TOKEN_LITERAL	NO_SUBTYPE_VALUE	" 'o"
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	hoLa
TOKEN_OPERATOR	OPERATOR_PLUS	+
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	hoLa
TOKEN_IDENTIFIER	NO_SUBTYPE_VALUE	hoLa
TOKEN_OPERATOR	OPERATOR_PLUS	+
TOKEN_LITERAL	NO_SUBTYPE_VALUE	"'sjas"

TOKEN TYPE	COUNT

TOKEN_KEYWORD	0
TOKEN_IDENTIFIER	5
TOKEN_PUNCTUATION	0
TOKEN_OPERATOR	2
TOKEN_CONSTANT	2
TOKEN_LITERAL	4

TOTAL VALID TOKENS	13

Figure 18: Output of Test 7.

5 Conclusions

The development of the lexical analyzer showed how the theory of regular languages is applied directly to a real problem. Each token category (**keywords**, **identifiers**, **constants**, **literals**, **operators**, and **punctuation**) was first described using a deterministic finite automaton and a regular expression, and the generated scanner behaves as the union of those automata. The results confirm that the classification of the input depends on two rules: the longest match and the order in which the patterns are declared. Both rules explain why `int` is a keyword while `integer` is an identifier, why `==` is a single operator, and why `efmemf.2121` is divided into an invalid identifier and a constant.

The tests also showed that the analyzer transforms a sequence of characters into an organized sequence of tokens, where each one contains its category, a specific attribute (such as the type of operator or error), and its lexeme. Whitespace is consumed without generating tokens, and malformed elements are classified into specific error types and reported without stopping the analysis, allowing all lexical errors in an input to be detected in a single execution. The analyzer works with text strings as well as files, which makes it usable with complete programs and not only with isolated examples.



The separation of the application into modules (scanner specification, token definitions, and main program) keeps the lexical rules independent from the handling of input and output, so the language can be extended by adding or modifying individual patterns.

Finally, the Context-Free Grammars proposed for the reading examples show how lexical analysis connects with the next phase of a compiler: the tokens produced by the lexer are the terminal symbols of the grammar. The left factoring applied to the declaration and the list of arguments removes the common prefixes, allowing a top-down parser to choose each production by looking at a single token. In this way, the project demonstrates why lexical analysis is the foundation on which syntactic analysis is built.

References

- [1] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.
- [2] Peter Bumbulis and Donald D. Cowan. “RE2C: A More Versatile Scanner Generator”. In: *ACM Letters on Programming Languages and Systems* 2.1–4 (1993), pp. 70–84.
- [3] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Boston, MA, USA: Pearson Addison-Wesley, 2006.
- [4] *re2c: Lexer Generator for C, C++, Go and Rust*. URL: <https://re2c.org> (visited on 10/06/2026).